```
    return [
      { id: 1, value: 10, valid: true },
      { id: 2, value: 20, valid: false },
      { id: 3, value: 15, valid: true }
    ];
  }
};
class DataSource { #B
  constructor(apiClient) {
    this.apiClient = apiClient;
  }
  async fetchData() {
    return await this.apiClient.get('/data');
  }
}
class DataSink { #C
  async saveData(processed) {
    console.log('Saved result:', processed);
  }
}
const cleanData = data => data.filter(item => item.valid); #D
const transformData = data => data.map(item => ({ ...item, value:
const summarizeData = data => ({ #F
  count: data.length,
  sum: data.reduce((acc, item) => acc + item.value, 0)
});
const pipe = (...fns) => x => #G
  fns.reduce((v, f) => f(v), x);
const processPipeline = pipe( #H
  cleanData,
  transformData,
  summarizeData
);
async function runPipeline() { #I
  const source = new DataSource(apiClient);
  const sink = new DataSink();
  const rawData = await source.fetchData();
  const result = processPipeline(rawData);
  await sink.saveData(result);
}
runPipeline(); #J
```

For this topic, are you able to answer these follow-up/probing questions?

"What are the advantages of using pipe over chaining methods directly (e.g., `data.map().filter().reduce()`)?"